

Step 9 Project Report: Heart Disease MLOps Pipeline

1. Executive Summary

This project delivers an end-to-end MLOps pipeline for a heart disease prediction system based on the UCI Cleveland heart disease dataset. It includes:

- data ingestion and preprocessing
- model training with multiple classifiers, hyperparameter tuning, and evaluation
- MLflow-compatible experiment logging
- FastAPI-based prediction service
- Docker packaging and local Kubernetes deployment
- Prometheus/Grafana monitoring for the API
- GitHub Actions CI for linting, testing, and smoke validation

The repository is structured to support reproducible experiment tracking, modular code, and infrastructure-as-code deployment.

2. Dataset and Problem Statement

The dataset is the UCI Cleveland heart disease dataset. The task is binary classification: predict whether a patient has heart disease ($\text{target} > 0$) or not.

Key features include:

- age, sex, chest pain type (`cp`)
- resting blood pressure (`trestbps`)
- serum cholesterol (`chol`)
- fasting blood sugar (`fbs`)
- resting electrocardiographic results (`restecg`)
- maximum heart rate achieved (`thalach`)
- exercise-induced angina (`exang`)
- ST depression induced by exercise (`oldpeak`)
- slope of peak exercise ST segment (`slope`)
- number of major vessels colored by fluoroscopy (`ca`)
- thalassemia status (`thal`)

The pipeline processes these features and trains a classifier capable of binary heart disease prediction.

3. Architecture Overview

3.1 Components

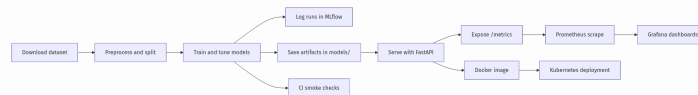
- `data/raw/heart.csv` - raw dataset source
- `notebooks/eda.ipynb` / `notebooks/eda_executed.ipynb` - exploratory data analysis, feature distributions, correlation and missing values analysis
- `src/data_preprocessing.py` - cleaning, imputation, encoding, and train/test split
- `src/preprocessing_pipeline.py` - scikit-learn `ColumnTransformer` pipeline for numeric and categorical features
- `src/train.py` - model training, hyperparameter search, evaluation, model persistence
- `src/api.py` - FastAPI service with prediction endpoint and Prometheus metrics
- `Dockerfile` - container packaging for the API
- `k8s/deployment.yaml` / `k8s/service.yaml` - Kubernetes deployment and service manifests
- `monitoring/prometheus.yml` - Prometheus scrape configuration

- `monitoring/grafana/dashboard.json` - Grafana dashboard definition
- `.github/workflows/ci.yml` - CI pipeline

3.2 Workflow Diagram

The core workflow is:

1. Download raw data with `scripts/download_data.py`
2. Clean and preprocess data with `src/data_preprocessing.py`
3. Train models in `src/train.py`, save best model and preprocessor to `models/`
4. Start API from `src/api.py` or package into Docker image
5. Deploy to Kubernetes using `k8s/` manifests
6. Collect runtime metrics with Prometheus and visualize via Grafana



Workflow Diagram (Mermaid)

4. Exploratory Data Analysis (EDA)

The EDA notebooks perform an in-depth review of the raw dataset before modeling. Key analysis includes:

- missing value assessment and imputation strategy justification
- class balance review for target labels
- feature distribution histograms for numeric variables
- correlation heatmap analysis to reveal linear relationships and possible multicollinearity
- selected pairplots to inspect relationships between key features and the target
- preprocessing decisions driven by observed feature patterns

The generated artifacts include `screenshots/histograms.png`, `screenshots/class_distribution.png`, `screenshots/correlation_heatmap.png`, and `screenshots/pairplot_selected.png`.

5. Data Preprocessing and Feature Engineering

5.1 Data cleaning

- raw values are loaded with `pandas`
- missing values marked as `?` are converted to `pd.NA`
- numeric columns are converted to numeric types when possible
- duplicate rows are removed

4.2 Imputation

- numeric features: median imputation
- categorical features: most frequent value imputation

4.3 Encoding

- categorical features are one-hot encoded with `OneHotEncoder(handle_unknown='ignore')`
- numeric features are scaled using `StandardScaler`

4.4 Persistence

- The fitted preprocessing pipeline is saved as `models/preprocessor.joblib`
- Processed splits are saved under `data/processed/`

5. Model Training and Evaluation

5.1 Models considered

The training pipeline builds and evaluates multiple models.

- Logistic Regression
- Random Forest
- XGBoost / Gradient Boosting
- Support Vector Classifier (SVC)

The pipeline supports both grid search and randomized search.

5.2 Training process

- `src/train.py` defines `train_and_log()` and `train_from_csv()` entry points
- models are evaluated with cross-validation
- metrics logged include accuracy, precision, recall, F1 score, and ROC AUC when available
- model artifacts are saved as `models/model_<name>.joblib` and `models/best_model.joblib`
- when MLflow is available, runs are logged locally under `mlruns/` and can be viewed with `python -m mlflow ui --host 127.0.0.1 --port 5001`

5.3 Evaluation artifacts

The repository contains evaluation artifacts generated by `scripts/generate_eval_plots.py`:

- `screenshots/roc_curves.png`
- `screenshots/pr_curves.png`
- `screenshots/confusion_best_model.png`
- `screenshots/confusion_model_logreg.png`
- `screenshots/confusion_model_rf.png`
- `screenshots/confusion_model_svc.png`

These plots show model discrimination and classification performance across candidate models.

5.4 Experiment tracking summary

- MLflow logging is enabled in training and preprocessing when the `mlflow` package is available.
- Runs are stored in the local tracking directory: `mlruns/`.
- Training logs model metrics (accuracy, precision, recall, F1, ROC AUC), model artifacts, and selected plots.
- Local UI command:

```
.venv\Scripts\python.exe -m mlflow ui --host 127.0.0.1 --port 5001
```

- Local UI URL: `http://127.0.0.1:5001`

MLflow UI screenshot (Experiments view):

Run Name	Created	User	Source	Models	accuracy	best_accuracy	f1	precision	recall	roc
inc	3 hours ago	roope	Df-c	model_inc	0.92	-	0.904791894...	0.95	0.963030363...	0.9
ygb	3 hours ago	roope	Df-c	model_ygb	0.9	-	0.879048789...	0.947598421...	0.918181818...	0.9
rf	3 hours ago	roope	Df-c	model_rf	0.94	-	0.929827828...	1	0.963030363...	0.9
lgbm	3 hours ago	roope	Df-c	model_lgbm	0.94	-	0.929827828...	1	0.963030363...	0.9
best_model_registry	3 hours ago	roope	Df-c	heart_disease_best_model...	-	0.94	-	-	-	-
inc	3 hours ago	roope	Df-c	model_inc	0.92	-	0.904791894...	0.95	0.963030363...	0.9
ygb	3 hours ago	roope	Df-c	model_ygb	0.9	-	0.879048789...	0.947598421...	0.918181818...	0.9
rf	3 hours ago	roope	Df-c	model_rf	0.94	-	0.929827828...	1	0.963030363...	0.9
lgbm	3 hours ago	roope	Df-c	model_lgbm	0.94	-	0.929827828...	1	0.963030363...	0.9
inc	3 hours ago	roope	Df-c	model_inc	0.903785123...	-	0.777251194...	0.82	0.738798718...	0.9
ygb	3 hours ago	roope	Df-c	model_ygb	0.769991735...	-	0.77532188...	0.737764618...	0.810810810...	0.9
rf	3 hours ago	roope	Df-c	model_rf	0.803785123...	-	0.787533314...	0.790909090...	0.787878787...	0.9
lgbm	3 hours ago	roope	Df-c	model_lgbm	0.818181818...	-	0.78203283...	0.811881188...	0.787878787...	0.9
Dfloop...	10 hours ago	roope	Df-loop...	model_inc	0.9	-	0.919254327...	0.9	0.931044882...	0.9

MLflow Experiments UI

6. API Design and Runtime Monitoring

6.1 FastAPI service

The prediction API is implemented in `src/api.py` and exposes:

- `POST /predict` - accepts input JSON with features: `list`
- `GET /metrics` - Prometheus scrape endpoint for runtime metrics

During startup, the service loads:

- `models/best_model.joblib`
- `models/preprocessor.joblib`

The `/predict` endpoint accepts raw feature vectors in the expected column order and either applies the saved preprocessor or assumes the client already provided transformed features.

6.2 Prometheus metrics

The application exports three custom metrics:

- `heart_disease_api_requests_total{method,path,status_code}`
- `heart_disease_api_request_latency_seconds{method,path}`
- `heart_disease_api_errors_total{method,path,status_code}`

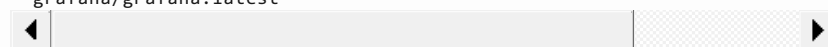
The middleware records request latency, request volume, and error count for all incoming HTTP traffic.

6.3 Monitoring manifests

- `monitoring/prometheus.yml` configures Prometheus to scrape `/metrics`
- `monitoring/grafana/dashboard.json` defines a Grafana dashboard with:
 - request rate timeseries
 - 95th-percentile latency
 - error rate
 - request rate by path

Minimal Grafana run command (auto-load provisioning on startup):

```
docker run --rm -p 3000:3000 --name grafana \
  -v "${PWD}/monitoring/grafana/provisioning:/etc/grafana/provisioning" \
  -v "${PWD}/monitoring/grafana/dashboard.json:/var/lib/grafana/dashboards
  grafana/grafana:latest
```



This mount setup auto-loads:

- Prometheus datasource from

- dashboard JSON from `monitoring/grafana/dashboard.json`

- the Grafana dashboard file contains PromQL panels for request rate, p95 latency, error rate, and request rate by path against those exported metrics

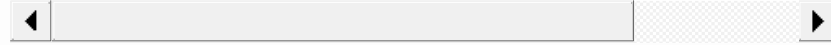
[illegible]

Prometheus Metrics Endpoint (Latest)

6.5 Executed Grafana run command and dashboard proof

Executed command:

```
docker run -d -p 3000:3000 --name grafana \
-v "${PWD}/monitoring/grafana/provisioning:/etc/grafana/provisioning" \
-v "${PWD}/monitoring/grafana/dashboard.json:/var/lib/grafana/dashboards
grafana/grafana:latest
```



Runtime proof captured:

- container status: grafana Up About a minute 0.0.0.0:3000->3000/tcp
- health endpoint: GET http://127.0.0.1:3000/api/health returned:

```
{
  "database": "ok",
  "version": "13.1.0",
  "commit": "b309c9bb3b81a748c3a75289236a27309ed2566a"
}
```

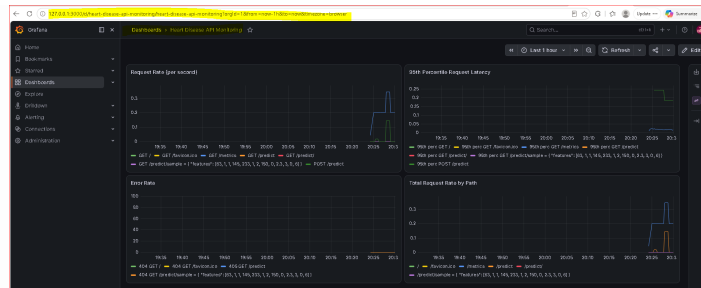
Grafana healthcheck endpoint screenshot:

```
Pretty-print
{
  "database": "ok",
  "version": "13.1.0",
  "commit": "b309c9bb3b81a748c3a75289236a27309ed2566a"
}
```

Grafana Healthcheck Endpoint

- Grafana datasource API returned provisioned Prometheus datasource (uid prometheus, url http://host.docker.internal:9090, isDefault: true)
- Grafana dashboard search API returned loaded dashboard:
 - uid: heart-disease-api-monitoring
 - title: Heart Disease API Monitoring
 - url: /d/heart-disease-api-monitoring/heart-disease-api-monitoring
- Grafana startup logs include provisioning events:
 - inserting datasource from configuration name=Prometheus uid=prometheus
 - starting to provision dashboards
 - finished to provision dashboards

Latest Grafana dashboard screenshot (live metrics):



Grafana Dashboard

7. Containerization and Kubernetes Deployment

7.1 Docker packaging

The Dockerfile builds a minimal Python container with all dependencies from requirements.txt.

Build command:

```
docker build -t heart-disease-api:latest .
```

Run locally:

```
docker run --rm -p 8000:8000 heart-disease-api:latest
```

7.2 Kubernetes deployment

The Kubernetes manifests are stored in k8s/.

Deploy the API with:

```
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
```

The service exposes the app on port 80 and routes traffic to container port 8000.

Deployment mode used in this project:

- local Kubernetes deployment (Minikube/Docker Desktop) is documented and supported.
- manifest-based deployment is used (k8s/deployment.yaml, k8s/service.yaml), which satisfies the deployment-manifest requirement.
- service exposure is configured via LoadBalancer in k8s/service.yaml.
- optional Ingress manifest is available in deployment/ingress.yaml when ingress-based exposure is preferred.

7.3 Local K8s notes

For Windows local Kubernetes, using Minikube with the Docker Desktop driver is recommended. The imagePullPolicy: IfNotPresent allows a locally built image to be used after loading it into Minikube.

If deploying into Minikube, use:

```
minikube image load heart-disease-api:latest
```

This avoids ErrImagePull for locally built images.

7.4 Endpoint verification after deployment

Use these checks after deploying to Kubernetes:

```
kubectl get pods
kubectl get svc heart-disease-api
kubectl get ingress
```

API endpoint verification examples:

```
curl http://127.0.0.1:8000/metrics
curl -X POST http://127.0.0.1:8000/predict -H "Content-Type: application/j
```

Expected verification evidence:

- /metrics responds with Prometheus series such as heart_disease_api_requests_total.
- /predict responds with a JSON prediction payload.
- Example /predict response: { "prediction": 0, "confidence": 0.42300383253750073 }.

8. CI/CD and Quality Validation

8.1 GitHub Actions

The CI workflow is defined in .github/workflows/ci.yml. It performs:

- code checkout
- Python 3.10 setup
- dependency installation from requirements.txt
- linting with flake8 (--max-line-length=120)
- unit tests via pytest
- coverage enforcement via pytest --cov=src --cov-fail-under=70
- quick smoke training run using synthetic data
- MLflow DB logging during smoke run using MLFLOW_TRACKING_URI=sqlite:///\${{github.workspace}}/mlflow_ci.db
- MLflow DB validation using MlflowClient.search_experiments() and run count checks
- artifact upload of the models/ directory
- artifact upload of MLflow tracking store (mlflow_ci.db and mlruns/)

This pipeline validates both code style and core training functionality. It also preserves CI experiment-tracking evidence as downloadable MLflow database artifacts.

8.2 Testing and linting

- pytest is used for functional and unit tests
- flake8 enforces style constraints and catches syntax issues
- Unit-test proof: tests/test_preprocessing.py covers data preprocessing behavior and tests/test_model.py covers model training/prediction behavior.

8.3 Code coverage proof

- Coverage is integrated in CI via pytest --cov=src --cov-report=term-missing --cov-report=xml --cov-report=html --cov-fail-under=70.
- Coverage threshold is enforced at 70% in .github/workflows/ci.yml.
- Coverage artifacts are uploaded in CI as coverage-report:
 - coverage.xml
 - htmlcov/

Latest executed local proof command:

```
.venv\Scripts\python.exe -m pytest -q --cov=src --cov-report=term-missing
```

Observed proof output summary:

- tests: 19 passed
- total coverage: 79%
- gate check: PASS because 79% >= 70%

Module-level coverage snapshot from the run:

- src/api.py: 87%
- src/data_preprocessing.py: 76%
- src/evaluation.py: 85%
- src/predict.py: 89%
- src/preprocessing_pipeline.py: 100%
- src/train.py: 73%
- src/utils.py: 100%

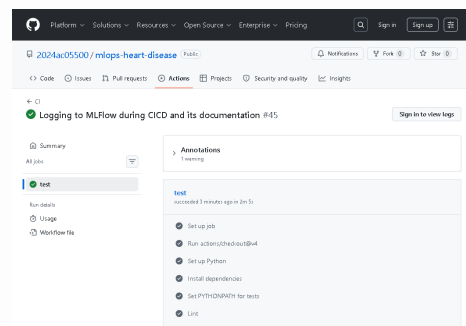
8.4 CI/CD and deployment workflow screenshots

- Add CI workflow run screenshots under screenshots/workflows/ (for example, lint/test/smoke stages from GitHub Actions).
- Add deployment workflow screenshots under screenshots/workflows/ (for example, Docker build, Kubernetes apply, service exposure).
- Reference files in this report after capturing them, for example:
 - screenshots/workflows/ci-run-summary.png
 - screenshots/workflows/docker-build-success.png
 - screenshots/workflows/k8s-deployment-status.png

Captured CI proof from:

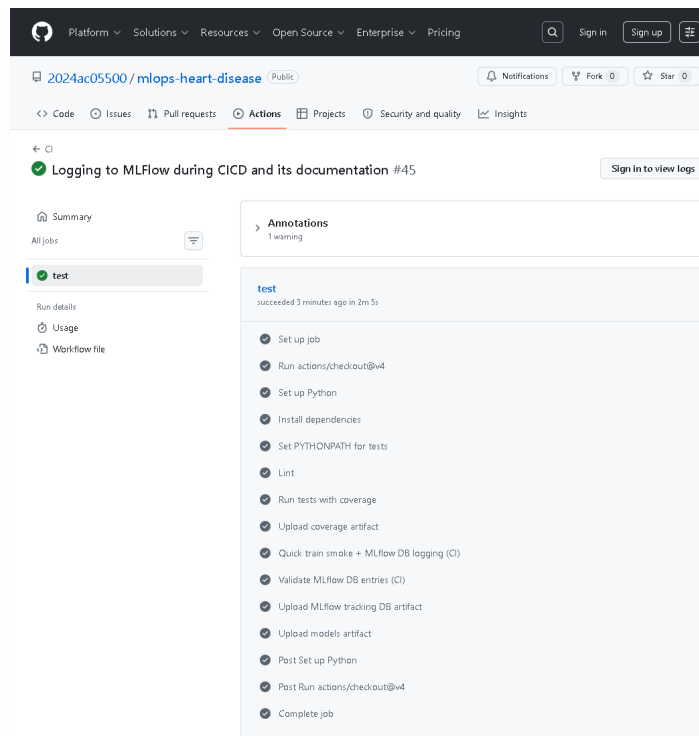
- <https://github.com/2024ac05500/mlops-heart-disease/actions/runs/29149667608/job/86537018128>

CI run summary screenshot:



CI Run Summary

CI step execution screenshot (includes lint, coverage, MLflow DB logging, MLflow DB validation, MLflow DB artifact upload, model artifact upload):

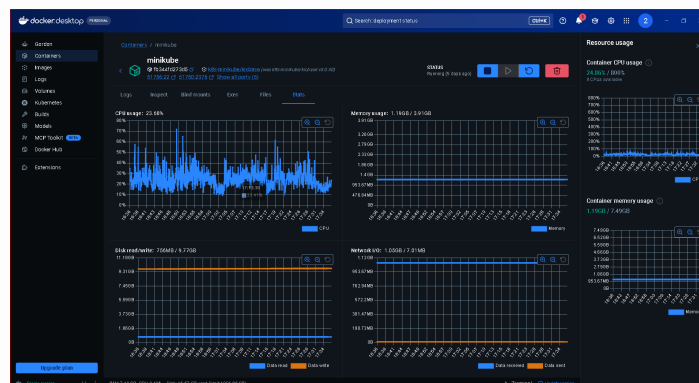


CI Step Execution Including MLflow DB Logging/Upload

Recommended deployment evidence screenshots for submission:

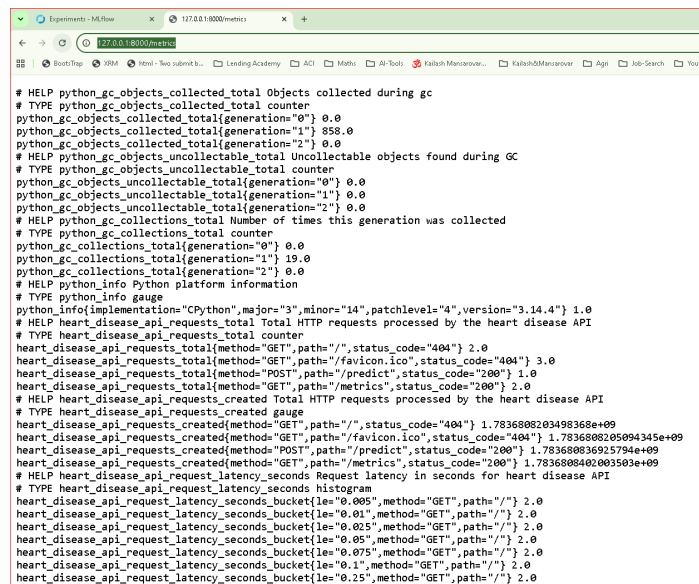
- screenshots/workflows/k8s-deployment-status.png (pods/deployment running)
- screenshots/workflows/deployed-api-metrics-endpoint.png (opened deployed /metrics endpoint)
- screenshots/workflows/deployed-api-predict-endpoint.png (successful deployed /predict response)

Kubernetes deployment status screenshot:



K8s Deployment Status

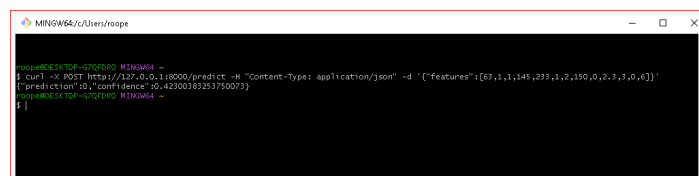
Deployed API metrics endpoint screenshot:



```
# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 0.0
python_gc_objects_collected_total{generation="1"} 858.0
python_gc_objects_collected_total{generation="2"} 0.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 0.0
python_gc_collections_total{generation="1"} 19.0
python_gc_collections_total{generation="2"} 0.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="14",patchlevel="4",version="3.14.4"} 1.0
# HELP heart_disease_api_requests_total Total HTTP requests processed by the heart disease API
# TYPE heart_disease_api_requests_total counter
heart_disease_api_requests_total{method="GET",path="/",status_code="404"} 2.0
heart_disease_api_requests_total{method="GET",path="/favicon.ico",status_code="404"} 3.0
heart_disease_api_requests_total{method="POST",path="/predict",status_code="200"} 1.0
heart_disease_api_requests_total{method="GET",path="/metrics",status_code="200"} 2.0
# HELP heart_disease_api_requests_created Total HTTP requests processed by the heart disease API
# TYPE heart_disease_api_requests_created gauge
heart_disease_api_requests_created{method="GET",path="/",status_code="404"} 1.7836808203498368e+09
heart_disease_api_requests_created{method="GET",path="/favicon.ico",status_code="404"} 1.7836808205094345e+09
heart_disease_api_requests_created{method="POST",path="/predict",status_code="200"} 1.783680836925794e+09
heart_disease_api_requests_created{method="GET",path="/metrics",status_code="200"} 1.7836808402003503e+09
# HELP heart_disease_api_request_latency_seconds Request latency in seconds for heart disease API
# TYPE heart_disease_api_request_latency_seconds histogram
heart_disease_api_request_latency_seconds_bucket{le="0.005",method="GET",path="/"} 2.0
heart_disease_api_request_latency_seconds_bucket{le="0.01",method="GET",path="/"} 2.0
heart_disease_api_request_latency_seconds_bucket{le="0.025",method="GET",path="/"} 2.0
heart_disease_api_request_latency_seconds_bucket{le="0.05",method="GET",path="/"} 2.0
heart_disease_api_request_latency_seconds_bucket{le="0.075",method="GET",path="/"} 2.0
heart_disease_api_request_latency_seconds_bucket{le="0.1",method="GET",path="/"} 2.0
heart_disease_api_request_latency_seconds_bucket{le="0.25",method="GET",path="/"} 2.0
```

Deployed API Metrics Endpoint

Deployed API predict endpoint screenshot:



```
MINGW64/c/Users/roope
$ curl -X POST http://127.0.0.1:8000/predict -H "Content-Type: application/json" -d '{"features": [61,1,1,145,233,1,2,150,0,2,3,3,0,6]}'
{"prediction": 0, "confidence": 0.42300183253750073}
$
```

Deployed API Predict Endpoint

9. Reproducibility and Environment

9.1 Dependency management

The repository includes both:

- requirements.txt for pip-based installation
- environment.yml for Conda-based reproducibility

environment.yml includes pinned versions for Python, scikit-learn, MLflow, Prometheus client, and other runtime libraries.

9.2 Reproducible model artifacts

- models/preprocessor.joblib preserves the exact transformation logic used during training
- models/best_model.joblib preserves the chosen predictive model

These artifacts ensure runtime prediction consistency with training preprocessing.

10. Usage Guide

10.1 Local environment setup

```
conda env create -f environment.yml
conda activate mlops-heart-disease
```

Or using pip:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

10.2 Data pipeline

```
python scripts/download_data.py
python -c "from src.data_preprocessing import load_csv, clean_df, preproc
```

10.3 Train models

```
python -c "from src.train import train_from_csv; train_from_csv('data/proc
```

10.4 MLflow UI

```
.venv\Scripts\python.exe -m mlflow ui --host 127.0.0.1 --port 5001
```

Open <http://127.0.0.1:5001> to inspect locally logged runs in mlruns/.

10.5 Run API

```
uvicorn src.api:app --reload --port 8000
```

10.6 Docker & Kubernetes

```
docker build -t heart-disease-api:latest .
minikube image load heart-disease-api:latest
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
```

10.7 Monitoring

- Prometheus config: monitoring/prometheus.yml
- Grafana dashboard: monitoring/grafana/dashboard.json
- API metrics endpoint: <http://127.0.0.1:8000/metrics> for local verification

11. Project Outcomes

This repository demonstrates a complete MLOps workflow including:

- data processing and feature engineering
- multi-model experimentation and evaluation
- runtime model serving with a production-ready API
- containerization and local Kubernetes deployment
- monitoring via Prometheus and Grafana
- automated CI quality checks

12. Key Learnings and Observations

- Reproducible model serving requires saving both the trained model and preprocessor.
- Local Kubernetes deployment on Windows is more reliable when the Docker Desktop driver is used and local images are loaded into the cluster.
- Prometheus middleware inside FastAPI enables lightweight observability for API performance and errors.
- CI should validate not only tests, but also a smoke training run to ensure the training pipeline remains executable.

13. References

- README.md
- Dockerfile
- k8s/deployment.yaml
- k8s/service.yaml
- monitoring/prometheus.yml
- monitoring/grafana/dashboard.json
- .github/workflows/ci.yml
- src/api.py
- src/train.py
- src/data_preprocessing.py
- src/preprocessing_pipeline.py
- scripts/download_data.py
- scripts/generate_eval_plots.py

14. Available Artifacts

- CI workflow: .github/workflows/ci.yml
- Docker image packaging: Dockerfile
- Kubernetes manifests: k8s/deployment.yaml, k8s/service.yaml
- Monitoring files: monitoring/prometheus.yml, monitoring/grafana/dashboard.json
- Visualization assets: screenshots/roc_curves.png, screenshots/pr_curves.png, and confusion matrix plots

15. Code Repository Link

- Repository URL: <https://github.com/2024ac05500/mlops-heart-disease>

16. YouTube Link

- YouTube Video URL : <https://youtu.be/FcXxGVCWVWI>